IOT & AI-BASED SMART AGRICULTURE PLATFORM FOR YIELD, DISEASE, IRRIGATION & PEST MANAGEMENT

COMMON INTEGRATED SOLUTION REPORT

Team member	Student ID	Individual component
V.G.S.M. Siriwardhana	IT22189158	Paddy Yield Forecasting
J.M.S.U.I. Bandara	IT22269348	Paddy Irrigation Optimization
A.P.N. Satyana	IT22205506	Disease Detection & Risk Forecasting
P.D.M. Kumarasinghe	IT22254566	Pest Management

Group ID: R26-IT-157

Report submitted in partial fulfilment of the requirements for the

B.Sc. (Hons) Degree in Information Technology

Specialising in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. We declare that this common report is our own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of our knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. We grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this report, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. We have disclosed all use of generative AI and other automated tools in accordance with the module policy. We remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this report.

Student name	Student ID	Signature	Date
V.G.S.M. Siriwardhana	IT22189158		10/11/2026
J.M.S.U.I. Bandara	IT22269348		10/11/2026
A.P.N. Satyana	IT22205506		10/11/2026
P.D.M. Kumarasinghe	IT22254566		10/11/2026

Supervisor certification. The above candidates carried out this research project under my supervision.

Role and name	Signature	Date
Supervisor: Ms. Thilini Jayalath		
Co-supervisor: Mr. Ashvinda Iddamalgoda		

ABSTRACT

Paddy farming in Sri Lanka is exposed to erratic monsoons, water stress, disease and pest pressure, yet smallholder farmers still decide on harvest planning, irrigation and crop protection mainly from experience and from reactive, disconnected information. This report presents the integrated solution of Group R26-IT-157, *The Digital Paddy*: an IoT and AI platform that treats yield uncertainty, irrigation timing, disease outbreaks and pest infestation as four interdependent subproblems of one decision problem. Four components share a common data backbone, API gateway and bilingual Sinhala–English delivery: an XGBoost yield forecaster (four to six weeks ahead), a stage-aware machine-learning irrigation scheduler, a ResNet-50 and LSTM disease detector and outbreak forecaster (seven- and fourteen-day risk), and a YOLOv8-based pest monitor with weather-driven hatching forecasts (seven to ten days).

Following a design-science approach, the components were built on low-cost Arduino/ESP32, ESP32-CAM and Raspberry Pi field hardware and integrated through shared JSON interfaces, so that signals such as soil wetness, outbreak alerts, spray windows and pest risk flow between modules instead of being captured and interpreted separately. The solution was piloted in the Negombo area during Yala 2026. The principal contribution is a coherent, interoperable decision-support architecture with an evaluation framework, which moves smallholder paddy management from separate, reactive tools towards coordinated, evidence-based action, and which relates directly to the Sustainable Development Goals on food security, water and climate adaptation.

Keywords: smart agriculture, paddy, IoT, machine learning, integrated decision support

ACKNOWLEDGEMENTS

We sincerely thank our supervisor, Ms. Thilini Jayalath, and our co-supervisor, Mr. Ashvinda Iddamalgoda, for their guidance, feedback and support throughout the project.

We are grateful to the Rice Research and Development Institute (RRDI) for historical records and domain references; to the agricultural experts at the Rice Research Station, Sammanthurai (Ampara) and RRDI, Batalagoda, who helped validate the pest data; to the Department of Agriculture agronomists and extension officers who reviewed disease information and advice; and to the paddy farmers of the Negombo area who gave field access and practical feedback during the Yala 2026 pilot.

TABLE OF CONTENTS

1 INTRODUCTION AND PROJECT CONTEXT 1
1.1 Background and context 1
1.2 Stakeholders and user needs 1
1.3 Alignment with the Sustainable Development Goals 2
1.4 Purpose and scope of the common report 3
2 OVERALL PROBLEM DEFINITION AND FOUR SUBPROBLEMS 4
2.1 Overall problem definition 4
2.2 Root causes and overall research gap 4
2.3 Subproblem and component mapping 5
2.4 Overall aim and group objectives 5
2.5 Scope assumptions and constraints 6
3 OVERALL SOLUTION STRUCTURE 8
3.1 Integrated solution overview 8
3.2 High level architecture 8
3.3 Component 1 summary 9
3.4 Component 2 summary 10
3.5 Component 3 summary 10
3.6 Component 4 summary 11
3.7 End to end user and data flow 11
4 COMPONENT INTEGRATION AND COHESIVE OPERATION 14
4.1 Integration strategy 14
4.2 Interfaces data contracts and shared models 14
4.3 Shared nonfunctional requirements 15
4.4 Integrated workflow and exception handling 16
4.5 Team contribution and dependency matrix 17
5 INTEGRATED IMPLEMENTATION AND PROJECT EXECUTION 18
5.1 Common technology stack and infrastructure 18
5.2 Development and integration process 18
5.3 Deployment and operational configuration 19
5.4 Evidence of team execution 19
6 INTEGRATED TESTING EVALUATION AND VALIDATION 21
6.1 Evaluation framework and success criteria 21
6.2 Integration and end to end testing 21
6.3 Overall results 22
6.4 Stakeholder and user validation 22
6.5 Achievement of common objectives 23
6.6 Integrated limitations and threats 23

7 COMMERCIALISATION SUSTAINABILITY AND RESPONSIBLE INNOVATION 25
7.1 Value proposition and beneficiaries 25
7.2 Market alternatives and differentiation 25
7.3 Business model and feasibility 26
7.4 Adoption intellectual property and compliance 27
7.5 Sustainable Development Goal contribution 27
8 CONCLUSION AND FUTURE DIRECTIONS 29
8.1 Overall conclusion 29
8.2 Principal integrated contribution 29
8.3 Future directions 29
REFERENCES 31
APPENDICES 33
Appendix A Integrated interface specification 33
Appendix B Team contribution declaration 33

LIST OF FIGURES

Figure 3.1 High-level architecture of the integrated solution showing users, shared services, the four components, external systems and deployment boundaries. 9
Figure 3.2 End-to-end data and user flow through the four components..... 12

LIST OF TABLES

Table 1.1 Stakeholders and user needs of the integrated solution 2
Table 1.2 Intended alignment with the Sustainable Development Goals 2
Table 1.3 Where detailed component evidence is found 3
Table 2.1 Four subproblems and their mapping to components 5
Table 2.2 Group objectives, responsible components and evaluation evidence 6
Table 3.1 Summary of Component 1: Paddy Yield Forecasting 9
Table 3.2 Summary of Component 2: Paddy Irrigation Optimization 10
Table 3.3 Summary of Component 3: Disease Detection and Risk Forecasting 10
Table 3.4 Summary of Component 4: Pest Management 11
Table 3.5 Cross-component exchanges (flows 1 to 10) 12
Table 4.1 Integration risks and mitigation 14
Table 4.2 Shared non-functional requirements 15
Table 4.3 Failure paths, detection and recovery 16
Table 4.4 Team contribution matrix 17
Table 4.5 Dependency matrix (flow numbers from Table 3.5) 17
Table 5.1 Common and component-specific technology stack 18
Table 5.2 Sources of evidence of team execution 20
Table 6.1 Group-level acceptance criteria, baselines and data 21
Table 6.2 Integrated test matrix and outcomes 21
Table 6.3 Overall results against component targets 22
Table 6.4 Achievement of the group objectives 23
Table 7.1 Alternatives and differentiation 25
Table 7.2 Combined pilot budget from the component theses 26
Table A.1 Integrated interface specification 33

LIST OF ABBREVIATIONS

Abbreviation	Meaning
ANN	Artificial Neural Network
API	Application Programming Interface
AUC	Area Under the ROC Curve
CNN	Convolutional Neural Network
DOA	Department of Agriculture
EC	Electrical Conductivity
ETL	Extract, Transform and Load
GDD	Growing Degree Days
IoT	Internet of Things
JSON	JavaScript Object Notation
LSTM	Long Short-Term Memory
mAP	mean Average Precision
ML	Machine Learning
REST	Representational State Transfer
RMSE	Root Mean Squared Error
RRDI	Rice Research and Development Institute
R^2	Coefficient of determination
SDG	Sustainable Development Goal
SLIIT	Sri Lanka Institute of Information Technology
UI	User Interface
XGBoost	eXtreme Gradient Boosting
YOLO	You Only Look Once

1 INTRODUCTION AND PROJECT CONTEXT

1.1 Background and context

Rice is the staple food of Sri Lanka, and paddy is grown by large numbers of smallholder farmers in two main seasons, Maha (roughly September to March) and Yala (roughly May to August). Production depends on rainfall and irrigation water, and both are increasingly erratic. Studies of Sri Lankan paddy show that yield is closely related to climatic variables such as humidity and temperature [1], [2], which means that a poorly timed monsoon, a dry spell or a humid spell can change the outcome of a season.

At the same time, the farmer faces four recurring decisions on the same field: how much to expect at harvest, when and how much to irrigate, whether a disease outbreak is coming, and whether a pest infestation is building up. In practice these decisions are still taken largely from experience. Irrigation follows canal rotations and routine rather than field evidence, and threshold-based sensor systems, although better than fixed schedules, are reactive and do not learn how water demand changes with growth stage [3]. Rice blast and sheath blight are usually recognised only after symptoms have spread, and losses of 20–40% in humid monsoon periods are reported in the disease component's analysis [4]. Pest control is likewise reactive: by the time larvae are seen feeding, preventive and low-impact treatment is no longer possible, and heavy chemical use follows [5].

Extension services cannot continuously monitor every smallholding, and existing digital tools address one problem at a time. This report therefore addresses the farmer's decision problem as a whole, through one platform in which field sensing, imagery, weather data and expert knowledge are captured once and used by four cooperating components.

1.2 Stakeholders and user needs

The primary stakeholders are smallholder paddy farmers in the Negombo pilot area, typically with a smartphone, intermittent connectivity and a strong preference for guidance in Sinhala. Secondary stakeholders are the officers and institutions that

advise them or use the data. Table 1.1 summarises their needs as recorded in the four component theses [6], [3], [4], [5].

Table 1.1 Stakeholders and user needs of the integrated solution

Stakeholder	Role	Principal needs
Smallholder farmers	Primary users	Simple, timely recommendations rather than raw data; alerts and advice in Sinhala and English; ability to override automated advice; low cost.
Agrarian and extension officers	Supervision, advisory support	Regional risk summaries, field history and seasonal reports to prioritise visits.
DOA agronomists	Expert review	Evidence-backed disease and pest recommendations and a feedback loop for corrections.
RRDI researchers and research stations	Data providers and reviewers	Visibility of model accuracy, access to field data, review of forecasts and predictions.
Farmer cooperatives and institutions	Potential customers and partners	District-level analytics and cluster purchasing of low-cost hardware.
Project team and administrators	Operation and maintenance	Reliable shared APIs, sensor and model maintenance, secure data handling.

Component needs were elicited through the individual studies, with expert consultation reported at the Rice Research Station, Sammanthurai and RRDI, Batalagoda for pest data [5], and with Department of Agriculture agronomists for disease labels and advice [4].

1.3 Alignment with the Sustainable Development Goals

The solution is aligned primarily with **SDG 2 (Zero Hunger)**, because its purpose is to protect and stabilise smallholder paddy production. Secondary alignments are listed in Table 1.2. These are intended contributions that follow from the design; their measured effect is discussed in Section 7.5, where possible negative effects are also considered.

Table 1.2 Intended alignment with the Sustainable Development Goals

SDG	Intended contribution	Main components
SDG 2 Zero Hunger	More stable smallholder yields through earlier forecasts, disease and pest warnings, and better water timing	All four
SDG 6 Clean Water and Sanitation	More efficient use of irrigation water through stage-aware scheduling	Irrigation, yield
SDG 12 Responsible Consumption and Production	Targeted rather than calendar-based spraying; less waste of water and agrochemicals	Disease, pest, irrigation

SDG	Intended contribution	Main components
SDG 13 Climate Action	Adaptation to monsoon and humidity variability through forecasting	Yield, disease, pest
SDG 9 Industry, Innovation and Infrastructure	Low-cost IoT and AI infrastructure for smallholder agriculture	All four
SDG 1 No Poverty	Protecting smallholder income by limiting avoidable loss and input cost	Disease, pest

1.4 Purpose and scope of the common report

This common report presents the integrated problem and solution of Group R26-IT-157: the shared problem, its four subproblems, the architecture and integration of the four components, the group-level evaluation framework, and the commercial and sustainability case. It does not repeat the component studies. The detailed research, development and test evidence for each component is in the four individual theses [6], [3], [4], [5], and this report refers to them wherever component-level detail is needed.

Table 1.3 Where detailed component evidence is found

Component	Student	Individual thesis (detailed methods, experiments, results)
1 Yield forecasting	V.G.S.M. Siriwardhana (IT22189158)	[6]
2 Irrigation optimisation	J.M.S.U.I. Bandara (IT22269348)	[3]
3 Disease detection and risk forecasting	A.P.N. Satyana (IT22205506)	[4]
4 Pest management	P.D.M. Kumarasinghe (IT22254566)	[5]

2 OVERALL PROBLEM DEFINITION AND FOUR SUBPROBLEMS

2.1 Overall problem definition

Problem. Sri Lankan smallholder paddy farmers must make four interdependent decisions on yield planning, irrigation, disease control and pest control, but they lack affordable, field-level, locally validated and timely decision support. They rely on experience and on separate, mostly reactive tools, so interventions come after damage has begun, water and agrochemicals are used inefficiently, and yield is lost that earlier action could have protected.

Stated as a question: *how can field-level sensing, imagery and weather data be combined, once and consistently, into early and mutually consistent forecasts and recommendations for yield, irrigation, disease and pests that a smallholder can receive in his or her own language and act on?*

2.2 Root causes and overall research gap

Four root causes recur across the component studies:

- **Reliance on experience and calendars.** Irrigation by canal turn, spraying by calendar or after symptoms, and harvest planning by intuition do not use field evidence [3], [5].
- **Reactive tools.** Threshold irrigation, mobile leaf-diagnosis applications and trap alarms act after a condition has appeared and do not forecast [3], [4], [5].
- **Isolated, non-local models.** Yield models rely on historical weather or satellite data and rarely use live plot-level soil data for Sri Lankan varieties [6]; disease and pest tools are often trained on global or laboratory data [4], [5].
- **Fragmentation.** Each decision tool has its own data capture and interface, so context that one problem generates for another (soil wetness, spray timing, infection pressure) is lost.

The overall research gap is therefore the absence of a low-cost, locally validated platform in which these four decision engines share one data backbone and exchange

the context that links them, delivered in the farmer's language. Closing it requires four components working together; no single component can address it alone.

2.3 Subproblem and component mapping

The common problem is divided into four subproblems that do not overlap in method or data ownership but depend on one another (Table 2.1).

Table 2.1 Four subproblems and their mapping to components

Subproblem	Question addressed	Component and owner	Why it is necessary
SP1 Yield uncertainty	How much will this field yield, four to six weeks before harvest?	Yield forecasting (Siriwardhana)	Gives the farmer and officers an early outlook for planning, and a common indicator of the cost of stress.
SP2 Water timing	When, and how urgently, should this plot be irrigated?	Irrigation optimisation (Bandara)	Water stress and over-watering affect yield and create disease-favourable conditions.
SP3 Disease outbreaks	Is a disease present, and how likely is an outbreak in the next 7–14 days?	Disease detection and risk forecasting (Satyana)	Outbreaks cause major loss and depend on humidity and wetness that irrigation influences.
SP4 Pest infestation	When will larvae hatch, and when is low-impact treatment most effective?	Pest management (Kumarasinghe)	Reactive control raises chemical use; treatment timing interacts with watering.

The dependencies are what make a four-component solution necessary. Soil moisture is both an irrigation output and a disease-risk input; spraying for disease or pests must be scheduled around watering so that treatments are not washed off; disease and pest pressure reduce the confidence of a yield forecast; and irrigation history and stress trends improve it. Treating the four subproblems in isolation would repeat sensing, miss these interactions and give the farmer conflicting advice.

2.4 Overall aim and group objectives

Aim. To design, build and pilot an interoperable IoT and AI platform that converts shared field sensing, imagery and weather data into early, consistent and actionable yield, irrigation, disease and pest recommendations for Sri Lankan smallholder paddy farmers, delivered in Sinhala and English.

The group objectives, the components responsible, and the evidence used to evaluate them are given in Table 2.2. Component-level targets are those set in the individual theses.

Table 2.2 Group objectives, responsible components and evaluation evidence

ID	Group objective	Components	Measure and evidence
GO1	Operate one platform in which the four components share a data backbone, API gateway and bilingual delivery	All	Interface tests and end-to-end tests (Section 6.2)
GO2	Forecast yield four to six weeks ahead	Yield	$R^2 \geq 0.85$ and RMSE < 0.5 t/ha [6]
GO3	Recommend irrigation timing and urgency and reduce water use	Irrigation	Validation accuracy above 90%; water saving of 30–50% against current practice [3]
GO4	Warn of disease outbreaks seven to fourteen days ahead	Disease	F1 ≥ 0.90; AUC ≥ 0.85; response under 3 s [4]
GO5	Forecast pest hatching seven to ten days ahead	Pest	mAP ≥ 90%; forecast error within ±2 days; response under 2 s [5]
GO6	Deliver usable, bilingual advice to farmers and validate it in a pilot	All	Farmer satisfaction of at least 80% in the Yala 2026 Negombo pilot
GO7	Operate responsibly: consent, anonymisation, secure data and transparent advice	All	Ethics and security checks (Sections 4.3, 7.4)

2.5 Scope assumptions and constraints

In scope: sensing and data capture for soil, weather, leaf images and trap images; the four decision engines; shared APIs and data stores; a bilingual farmer interface and officer dashboard; and a pilot in the Negombo area during Yala 2026 on the varieties and diseases and pests addressed by the components (BG 352 and BG 367 for yield; four leaf classes for disease; target pests such as Brown Plant Hopper for pest).

Out of scope: crops other than paddy; autonomous, unsupervised valve actuation or pesticide dosing; large commercial sensor networks; and diseases and pests outside the trained classes.

- **Assumptions.** Sensors can be calibrated and maintained; farmers have a smartphone and intermittent connectivity; external weather services (for example NASA POWER [7]) are available at the polling interval; and farmers consent to participate and report outcomes.

- **Constraints.** A single season (Yala 2026), one pilot area and a small number of nodes (three sensor stations or traps per component) limit the generality of the evidence; the budget is a modest pilot budget (Section 7.3); and the farmer-facing interface must work in Sinhala and English.
- **Dependencies.** Yield, irrigation, disease and pest outputs consume each other's data, so an unavailable or invalid upstream input must degrade gracefully (Section 4.4).

3 OVERALL SOLUTION STRUCTURE

3.1 Integrated solution overview

The solution, *The Digital Paddy*, is one decision-support platform rather than four applications. Its value proposition is a single path from field evidence to farmer action: sensors, cameras and farmer photographs feed a shared backbone; four engines forecast yield, schedule irrigation, and warn of diseases and pests while sharing the context that links them; and the results reach farmers in Sinhala and English and reach officers and researchers through dashboards and reports.

Three design principles hold it together. **Capture once, use many times:** each measurement (for example soil moisture or humidity) is stored in the shared backbone and reused by every component that needs it. **Interface first:** components exchange data only through versioned JSON contracts, so they can be developed and tested independently and still work together. **Farmer in control:** advice is presented with confidence levels and explanations, and the farmer can override automated recommendations; sensor faults trigger a request for manual inspection rather than silent failure.

3.2 High level architecture

Figure 3.1 shows the architecture. Field and edge devices at the pilot plots capture data: soil and water sensor nodes (Arduino/ESP32), a weather station, farmer smartphone photographs, ESP32-CAM pheromone traps and a Raspberry Pi 4 edge node that buffers data and supports local inference when connectivity is poor. Data pass through a shared API gateway, which authenticates requests, validates and time-aligns data and exposes JSON interfaces, into shared data stores (Firebase for real-time synchronisation and PostgreSQL for time series and records) [6], [4]. The four analytics engines run in the cloud and exchange data through the gateway. External systems supply RRDI historical records, NASA POWER and meteorological data and the Department of Agriculture treatment knowledge base. Results are delivered to the bilingual farmer app, the officer dashboard and reports.

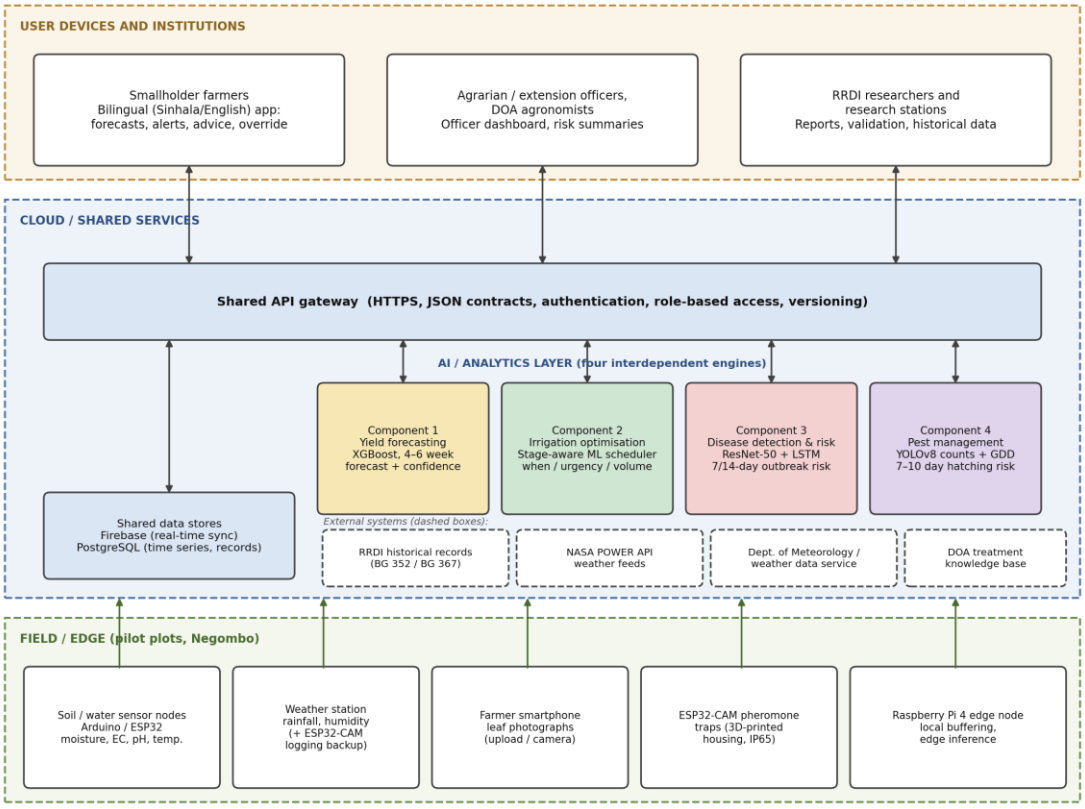


Figure 3.1 High-level architecture of the integrated solution showing users, shared services, the four components, external systems and deployment boundaries.

Three deployment boundaries are shown: the **field/edge** boundary (devices in the plots), the **cloud/shared services** boundary (gateway, data stores and analytics) and the **user devices and institutions** boundary (farmer phones, officer dashboards and research users). Data crossing a boundary is authenticated and sent over HTTPS, and records carry anonymised plot identifiers.

3.3 Component 1 summary

Component 1 is the Paddy Yield Forecasting component [6]. It addresses subproblem SP1 and provides the platform's early view of expected yield. Table 3.1 summarises it.

Table 3.1 Summary of Component 1: Paddy Yield Forecasting

Attribute	Summary
Owner	V.G.S.M. Siriwardhana (IT22189158)
Subproblem	SP1: yield uncertainty, four to six weeks before harvest
Inputs	Soil moisture, EC, pH and temperature from field stations; rainfall and humidity; RRDI historical yield records for BG 352 and BG 367; companion-module signals

Attribute	Summary
Core method	Pre-processing and feature engineering for monsoon intensity and growth stage; XGBoost regressor [8] benchmarked against Random Forest [9] and ANN; five-fold cross-validation and soil/weather ablation
Outputs	Yield forecast (t/ha), confidence score and risk flag as JSON and in the farmer interface
Targets	$R^2 \geq 0.85$; RMSE < 0.5 t/ha; inference under 2 s
Contribution to the whole	Gives an early planning outlook; consumes irrigation history, disease risk and pest risk to adjust confidence; provides yield-risk information to irrigation

3.4 Component 2 summary

Component 2 is the Paddy Irrigation Optimization component [3]. It addresses subproblem SP2 and is the platform's water-management engine. Table 3.2 summarises it.

Table 3.2 Summary of Component 2: Paddy Irrigation Optimization

Attribute	Summary
Owner	J.M.S.U.I. Bandara (IT22269348)
Subproblem	SP2: when, and how urgently, to irrigate
Inputs	Soil moisture, EC, pH and temperature (Arduino/ESP32 with DFRobot-compatible probes); rainfall, humidity and short-range forecast; farmer-entered irrigation events and crop stage; season
Core method	Stage-aware features; scikit-learn models (threshold baseline, decision tree, random forest, gradient boosting) [10]; time-aware validation
Outputs	Irrigation status (irrigate now, delay, monitor), urgency and expected water use; fallback alert for sensor faults; farmer manual override
Targets	Validation accuracy above 90%; 30–50% water saving against current practice; screens under 2 s; satisfaction of at least 80%
Contribution to the whole	Supplies soil-moisture and irrigation context to the disease, yield and pest components and adjusts watering around spray windows

3.5 Component 3 summary

Component 3 is the Disease Detection and Risk Forecasting component [4]. It addresses subproblem SP3 and is the platform's disease-intelligence service. Table 3.3 summarises it.

Table 3.3 Summary of Component 3: Disease Detection and Risk Forecasting

Attribute	Summary
Owner	A.P.N. Satyana (IT22205506)
Subproblem	SP3: disease identification and 7- to 14-day outbreak risk

Attribute	Summary
Inputs	Farmer leaf photographs; 14-day sequences of humidity, temperature and rainfall from sensors and meteorological feeds; recent severity signals; irrigation and soil-moisture context
Core method	ResNet-50 transfer learning [11] for four leaf classes; LSTM [12] on weather sequences with CNN severity as an input feature; alert thresholds (High above 0.70; Watch 0.40–0.69; Low below 0.40)
Outputs	Disease class and severity, 7- and 14-day outbreak probability, confidence, Sinhala/English treatment advice from the DOA knowledge base
Targets	F1 ≥ 0.90; AUC ≥ 0.85; response under 3 s; satisfaction above 80%
Contribution to the whole	Provides disease risk to yield and spray-window information to irrigation, and alerts to the pest component

3.6 Component 4 summary

Component 4 is the Pest Management component [5]. It addresses subproblem SP4 and is the platform's pest early-warning engine. Table 3.4 summarises it.

Table 3.4 Summary of Component 4: Pest Management

Attribute	Summary
Owner	P.D.M. Kumarasinghe (IT22254566)
Subproblem	SP4: pest infestation and the larval hatching window
Inputs	Images from ESP32-CAM pheromone traps; temperature, humidity and precipitation from the NASA POWER API [7]; crop-stress and disease context from other components
Core method	YOLOv8 object detection [13] to count adult target pests (biofix); Growing Degree Days phenology logic; risk matrix (High, Medium, Low); mitigation scheduling
Outputs	7- to 10-day hatching window, risk level and eco-friendly mitigation schedule, shown in a bilingual React dashboard and via API
Targets	mAP ≥ 90%; forecast error within ±2 days; response under 2 s
Contribution to the whole	Sends trap microclimate and risk scores to yield and spray schedules to irrigation; receives crop-stress, disease and microclimate context

3.7 End to end user and data flow

Figure 3.2 traces how data move through the complete solution. Raw measurements, photographs, trap images and farmer entries are captured in the field. The gateway validates them, aligns them in time, anonymises identifiers and buffers them if the network is down; the shared stores and feature store make them available to every component. Each engine produces its forecast or recommendation, and the ten cross-component exchanges listed in Table 3.5 let one component use what another has learned. Results are delivered to the farmer app, officer dashboard and reports. Farmer

actions, overrides, observed harvests and outbreaks are logged and fed back to retrain the models.

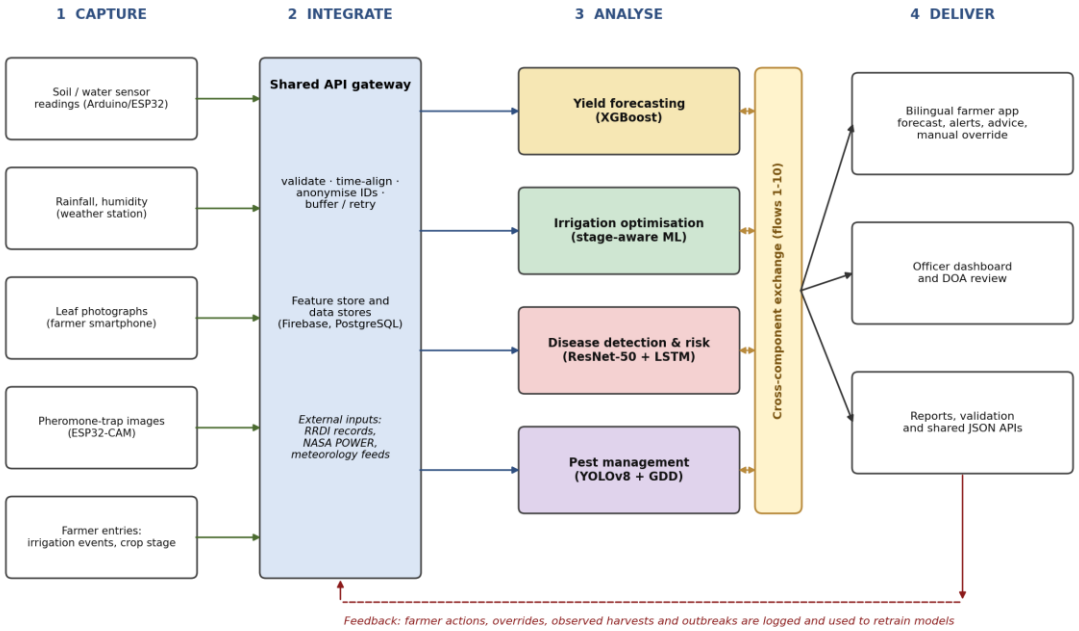


Figure 3.2 End-to-end data and user flow through the four components.

Table 3.5 Cross-component exchanges (flows 1 to 10)

No.	From → to	Information exchanged	Purpose
1	Irrigation → Yield	Irrigation history, water-stress trends, crop-stage data	Input features and confidence for the yield forecast
2	Yield → Irrigation	Yield-risk information	Supports water-scheduling decisions
3	Irrigation → Disease	Soil moisture and irrigation events	Wetness context for LSTM disease-risk prediction
4	Disease → Irrigation	Outbreak alert and spray/treatment window	Spray-window-aware watering adjustments
5	Disease → Yield	Severity and 7/14-day risk	Risk-adjusted yield outlook
6	Pest → Yield	Trap microclimate and pest-risk scores	Update yield forecast confidence
7	Pest → Irrigation	Pest threat level and spray schedule	Avoid wash-off of biopesticides
8	Irrigation → Pest	Field humidity and soil condition (microclimate history)	Context for pest-risk scoring
9	Disease → Pest	Cross-module alerts and disease presence	Refine pest vulnerability and coordinate advice
10	Yield → Pest	Crop-stress level	Refine pest vulnerability

For a farmer the flow is simple: a photograph or an automatic sensor reading enters the system, and a short recommendation comes back in Sinhala or English, for example

to irrigate within a day, to expect a high risk of blast in a week, or to apply a low-impact treatment on a stated day. Because the engines share context, these recommendations are consistent with one another.

4 COMPONENT INTEGRATION AND COHESIVE OPERATION

4.1 Integration strategy

Integration followed an interface-first strategy. The team first agreed which information each component must provide to the others (the flows in Table 3.5), then defined JSON contracts for them (Section 4.2) before the engines were finalised, so that each component could be built and tested against agreed interfaces and mock payloads. The producer of a payload owns its schema and version; consumers validate what they receive.

Integration proceeded in stages: (i) component-level tests in each student's development environment; (ii) pairwise interface tests against the shared gateway and data stores; (iii) end-to-end tests in a staging environment using recorded or simulated field data; and (iv) the pilot environment in the Negombo plots during Yala 2026. The principal integration risks and their mitigation are given in Table 4.1.

Table 4.1 Integration risks and mitigation

Risk	Mitigation
Schema drift between components	Versioned JSON contracts; producer-owned schemas; contract tests on every release
Inconsistent identifiers or units	Shared plot, node and timestamp identifiers; common unit table in Appendix A
Heterogeneous front ends and stores (Android/Flutter, React, Firebase, PostgreSQL)	Gateway as the single integration point; farmer-facing results exposed as the same JSON objects
External service outages (weather APIs)	Retries, cached last-known values and stale-data flags
Sensor faults and connectivity loss	Range and health checks; local buffering; fallback alert requesting manual inspection
Cascading errors between engines	Consumers treat upstream signals as optional context; forecasts carry confidence and flag missing inputs
Privacy of farmer data	Consent, anonymised IDs, role-based access, encrypted storage

4.2 Interfaces data contracts and shared models

All components communicate through REST-style HTTPS endpoints exchanging JSON. Every payload uses a common envelope so that identifiers, timing, provenance and confidence are handled consistently, and carries a component-specific body. An illustrative envelope is shown below

```
{
  "schema_version": "1.0",
  "component": "pest",
  "plot_id": "P-014",
  "generated_at": "2026-06-01T06:30:00+05:30",
  "model_version": "yolov8-v2",
  "confidence": 0.82,
  "inputs_complete": true,
  "language": "si",
  "body": { "risk_level": "High", "hatching_window_days": 8 }
}
```

- **Shared identifiers.** Anonymised plot identifier, node identifier, ISO 8601 timestamp with time zone, component name and model version.
- **Validation rules.** Required fields present; numeric values within physical ranges; timestamps not in the future; confidence between 0 and 1; payloads failing validation are rejected with an error code and logged.
- **Versioning.** Semantic versions for schemas and models; consumers accept the current and previous major versions during a release.
- **Error handling.** The gateway returns informative HTTP errors; producers retry with back-off; consumers fall back to the last valid value and set a stale flag.
- **Ownership.** Each component owns the schema of what it produces; the gateway and shared stores are maintained jointly

The complete interface specification, including payload fields, triggers and error handling for each exchange, is in Appendix A.

4.3 Shared nonfunctional requirements

The shared non-functional requirements follow the ISO/IEC 25010 quality characteristics [14] and are summarised in Table 4.2. Individual components add their own requirements in their theses.

Table 4.2 Shared non-functional requirements

Category	Requirement across the platform
Performance	Forecast and recommendation responses in under 2 s (yield, irrigation, pest); disease image classification in about 3 s or less
Usability and accessibility	Sinhala and English text and fonts on all farmer screens; simple single-screen results; confidence levels and plain-language advice; manual override
Security	HTTPS in transit; authentication and role-based access; encrypted storage; access-controlled imagery

Category	Requirement across the platform
Privacy	Informed consent; anonymised plot identifiers; location stored at field level only where needed; data used only for the project; compliance with [15]
Reliability	Local buffering at the edge and gateway; retries; fallback alerts for faulty or missing sensor data
Maintainability	Versioned models, schemas and APIs; repeatable retraining workflow; documented configuration
Scalability	Containerised cloud microservices and time-series storage designed to scale from a pilot to thousands of farms; staged hardware funding

4.4 Integrated workflow and exception handling

In normal operation the following sequence runs continuously. (1) Sensor nodes, the weather station and traps capture data and the farmer may upload leaf photographs and record irrigation events. (2) The gateway validates and stores them. (3) The engines run on a daily batch and are re-triggered when new data or companion signals arrive (for example a new trap image or a new disease alert). (4) Each engine publishes its result and the cross-component flows of Table 3.5. (5) The farmer and officers receive results, and the farmer may act or override. (6) Actions and observed outcomes are logged for retraining.

Important failure paths and the behaviour of the integrated solution are summarised in Table 4.3.

Table 4.3 Failure paths, detection and recovery

Failure	Detection	Behaviour and recovery
Sensor fault or missing readings	Range and health checks; missing or frozen values	Fallback alert requesting manual inspection; forecasts flag reduced confidence; no silent failure
Connectivity loss in the field	Failed uploads	Local buffering at the edge; synchronised on reconnection; edge inference where available
Weather API (for example NASA POWER) unavailable	Timeout or error response	Retries; last-known forecast used with stale flag; farmers informed that data are delayed
Upstream component output missing or invalid	Contract validation fails	Consumer continues without that context and reports lower confidence
Low-confidence or conflicting advice	Confidence below threshold or contradictory flags	Advice shown with confidence; medium-risk cases routed to officer or agronomist review

Failure	Detection	Behaviour and recovery
Unsafe actuation risk	Supervised pilot	Recommendations remain advisory; valve actions require manual confirmation and override

4.5 Team contribution and dependency matrix

Table 4.4 maps each student to a component, the research and development tasks, integration work and test evidence, and the shared outputs, as described in the individual theses [6], [3], [4], [5].

Table 4.4 Team contribution matrix

Student	Research and development tasks	Integration work
Siriwardhana (Yield)	Data collection and RRDI preparation; ETL and feature engineering; XGBoost and baseline benchmarking; forecast API	Consumes irrigation, disease and pest signals; supplies yield risk to irrigation
Bandara (Irrigation)	Sensor stack and calibration; data pipeline; stage-aware ML scheduler; recommendation service and mobile app with override	Supplies wetness and microclimate context; receives spray-window flags
Satyana (Disease)	Image and environmental data set; ResNet-50 classifier; LSTM risk model; alert logic	Supplies severity and risk; consumes irrigation wetness
Kumarasinghe (Pest)	ESP32-CAM trap hardware; annotated pest data set; YOLOv8; GDD risk logic; React dashboard	Supplies pest risk and spray schedules; consumes crop-stress and microclimate

Table 4.5 shows which component depends on which, by the flow numbers of Table 3.5. A filled cell means the column component consumes information from the row component.

Table 4.5 Dependency matrix (flow numbers from Table 3.5)

Provider ↓ / consumer →	Yield	Irrigation	Disease	Pest
Yield	–	2	–	10
Irrigation	1	–	3	8
Disease	5	4	–	9
Pest	6	7	–	–

Shared outputs (gateway, data stores, common contracts, farmer-interface integration and the common report) were produced jointly. Appendix B records the agreed contribution of each member.

5 INTEGRATED IMPLEMENTATION AND PROJECT EXECUTION

5.1 Common technology stack and infrastructure

Table 5.1 Common and component-specific technology stack

Layer	Technologies	Used by	Rationale
Field hardware	Arduino / ESP32 sensor nodes with DFRobot-compatible probes; weather station; ESP32-CAM traps; Raspberry Pi 4 edge node	All (by component)	Low-cost, available and suitable for pilot deployment
Connectivity	Wi-Fi / LoRa gateway; local buffering	Irrigation, yield, pest, disease	Tolerates intermittent rural connectivity
Data stores	Firebase (real-time synchronisation); PostgreSQL (time series and records)	All	Real-time updates to apps and reliable structured storage
Analytics	Python 3.11; scikit-learn [10]; XGBoost [8]; PyTorch and OpenCV; Ultralytics YOLOv8 [13]	By component	Open-source, mature ecosystems
Services	Docker microservices; cloud GPU for training; REST/JSON APIs	All	Reproducible deployment and scaling
Front ends	Android/Flutter mobile app (irrigation); React dashboard (pest); Sinhala/English mobile interface (yield, disease)	By component	Bilingual delivery to farmers and officers
External services	NASA POWER API [7]; meteorological data; RRDI records; DOA knowledge base	By component	Weather context, history and expert advice

Common choices were driven by cost (open-source software and off-the-shelf hardware), the need for offline tolerance in the field, and the need for a single set of interfaces. Where components use different front-end or storage technologies, the gateway keeps their outputs interoperable.

5.2 Development and integration process

The project was organised on a shared schedule with the Yala 2026 season as its deadline: data collection and hardware set-up, model development, API and app integration, and field validation (the component schedules in the individual theses run over weeks 10 to 24 [4]). The team followed a design-science approach [16], building each artefact, evaluating it and refining it.

Interface-first working (Section 4.1) meant that producers published example payloads early, so consumers could develop against them. Source code, models and documentation are version-controlled, and configuration (feature lists, seeds, hyperparameters) is kept with the code so that experiments can be reproduced.

5.3 Deployment and operational configuration

The pilot was deployed in paddy plots in the Negombo area during Yala 2026. Each component deployed its own field kit: three sensor stations and a weather station for yield (with an ESP32-CAM logging backup), three sensor node bundles for irrigation, three Raspberry Pi 4 edge nodes with USB cameras for disease, and ESP32-CAM pheromone traps for pest [6], [3], [4], [5]. Field devices upload to the shared backend, where services run as containers and are accessed only through the gateway.

- **Build and configuration.** Services are packaged with Docker and configured through environment and configuration files, with separate development, staging and pilot settings .
- **Monitoring.** Data availability, sensor health and service latency are monitored, and faults generate manual-inspection requests .
- **Backup and restore.** Data stores are backed up and models are versioned so that a previous model and schema can be restored .
- **Operation.** Models are retrained when new harvest or outbreak observations become available, and API and model versions are recorded.

5.4 Evidence of team execution

Evidence of how the team planned and carried out the work is held in the project's planning, communication and version-control records. Table 5.2 lists the sources; raw activity logs are not reproduced here.

Table 5.2 Sources of evidence of team execution

Source	What it shows
MS Planner	Task plan, ownership and progress by component
MS Teams	Meetings, decisions and supervisor feedback
Git history	Code contributions and integration commits by member
Meeting records and research diaries	Interface agreements, issues and changes

6 INTEGRATED TESTING EVALUATION AND VALIDATION

6.1 Evaluation framework and success criteria

The group evaluation answers four questions. **EQ1:** does each component meet the accuracy and performance targets set for it? **EQ2:** do the components interoperate end to end as designed? **EQ3:** do farmers and officers find the complete solution usable and useful in the pilot? **EQ4:** are the group objectives of Section 2.4 achieved? The acceptance criteria and baselines are in Table 6.1.

Table 6.1 Group-level acceptance criteria, baselines and data

Component	Metric and acceptance criterion	Baseline	Data and analysis
Yield	$R^2 \geq 0.85$; RMSE < 0.5 t/ha [6]	Weather-only features; RF and ANN	Five-fold cross-validation, soil/weather ablation, Yala 2026 harvest comparison
Irrigation	Accuracy above 90%; water saving 30–50% [3]	Threshold rule; current farmer practice	Time-aware validation; plot-level water use during Yala 2026
Disease	F1 ≥ 0.90; AUC ≥ 0.85; response under 3 s [4]	Weather-only LSTM	Held-out unseen-farm set; chronological splits
Pest	mAP ≥ 90%; forecast error within ±2 days; response under 2 s [5]	YOLOv5, Faster R-CNN; biofix-only forecasting	Annotated trap images; observed emergence timing
Integration	Interfaces 1–10 exchange valid payloads within the latency limit; graceful degradation	–	Interface, end-to-end and fault-injection tests
Users	Satisfaction of at least 80% of respondents	–	Pilot survey and action logs

6.2 Integration and end to end testing

Integration testing covered the interfaces in Table 3.5, the end-to-end path from field data to farmer screen, and the failure paths in Table 4.3. Table 6.2 lists the test matrix and records the outcomes.

Table 6.2 Integrated test matrix and outcomes

ID	Test	Expected result	Outcome
I1	Contract tests for flows 1–10 (valid and invalid payloads)	Valid payloads accepted; invalid rejected with error code	[Pass]
I2	End-to-end: sensor reading to yield forecast on farmer screen	Forecast displayed in Sinhala and English	[Pass]

ID	Test	Expected result	Outcome
I3	End-to-end: leaf photo to disease alert and spray-window flag to irrigation	Alert shown; irrigation advice adjusted	[Pass]
I4	End-to-end: trap image to pest risk and schedule to irrigation	Risk and schedule shown; watering adjusted	[Pass]
I5	Fault injection: sensor failure, API outage, connectivity loss	Fallback alerts; buffered data synchronised; stale flags	[Pass]
I6	Performance: response times under typical load	Within limits of Section 4.3	[Measured values]
I7	Security: unauthenticated and unauthorised access	Access refused; roles enforced	[Pass]
I8	Usability: Sinhala and English screens on target devices	Correct text, fonts and navigation	[Pass]
I9	Regression after model or schema update	Previous contracts still honoured	[Pass]

6.3 Overall results

Table 6.3 synthesises the group-level results against the targets. Detailed experiments, tables and figures for each component are in the individual theses, to which readers are referred [6], [3], [4], [5].

Table 6.3 Overall results against component targets

Indicator	Component	Target	Result
R^2 / RMSE of yield forecast	Yield	≥ 0.85 / < 0.5 t/ha	[0.65t/ha]
Irrigation validation accuracy	Irrigation	Above 90%	[95%]
Water saving against current practice	Irrigation	30–50%	[40%]
Disease classification F1 (held-out)	Disease	≥ 0.90	[0.8]
Outbreak forecast AUC	Disease	≥ 0.85	[0.76]
Pest detection mAP@0.5	Pest	≥ 90%	[87%]
Hatching-window forecast error	Pest	Within ±2 days	[Pass]
Response time (yield / irrigation / pest; disease)	All	< 2 s; < 3 s	[1s]

6.4 Stakeholder and user validation

The complete solution was evaluated in the Yala 2026 pilot by smallholder farmers in the Negombo area and by extension officers, with domain review by agronomists and RRDI researchers. Evidence was collected through a short satisfaction survey, action and override logs from the apps, comparison of alerts and forecasts with observed

outcomes, and expert review of disease and pest data and advice [4], [5]. The platform-level target is satisfaction of at least 80% of respondents.

6.5 Achievement of common objectives

Table 6.4 Achievement of the group objectives

ID	Group objective	Evidence
GO1	Integrated platform with shared backbone	Sections 4.2, 6.2; Appendix A
GO2	Yield forecast 4–6 weeks ahead	Table 6.3; [6]
GO3	Irrigation recommendations and water saving	Table 6.3; [3]
GO4	Disease outbreak warning 7–14 days ahead	Table 6.3; [4]
GO5	Pest hatching forecast 7–10 days ahead	Table 6.3; [5]
GO6	Usable bilingual advice validated in the pilot	Section 6.4
GO7	Responsible operation	Sections 4.3 and 7.4

6.6 Integrated limitations and threats

- **Small, single-season pilot.** Three nodes per component in one area during one season give few independent observations, so accuracy and water-saving estimates have wide uncertainty and may not generalise to other soils, varieties, seasons or districts.
- **Dependencies between components.** Flows 1–10 mean that an error or outage in one engine can affect others; the design limits this through optional context and confidence flags, but cascading effects are only partly testable in one season.
- **Interaction effects are hard to attribute.** When several engines change the advice at once, it is difficult to say which one produced a benefit; a controlled comparison with and without exchanges would be needed.
- **Sensor and hardware quality.** Low-cost probes, lenses and lures degrade, particularly in flooded fields and monsoon conditions, and microclimate in the canopy can differ from API weather.
- **Label and data quality.** Disease and pest labels depend on expert confirmation and trap placement; yield data depend on harvest reporting.

- **Uneven evidence across components.** The depth of validation is not identical across the four components
- **Unresolved integration risks.**

7 COMMERCIALISATION SUSTAINABILITY AND RESPONSIBLE INNOVATION

7.1 Value proposition and beneficiaries

For farmers, the solution replaces guesswork and four separate tools with one bilingual service that tells them what to expect, when to water, and what to watch for, early enough to act and with a manual override. For officers and agronomists it provides field-level evidence to prioritise visits and review advice. For RRDI researchers it provides structured field and outcome data. For society it supports food security, more efficient use of water and agrochemicals, and adaptation to climate variability. Stakeholder needs recorded in the component studies support these claims [6], [3], [4], [5]; .

7.2 Market alternatives and differentiation

The target segments are smallholder paddy farmers and farmer clusters first, then agrarian service centres, cooperatives and extension services as institutional users. Table 7.1 compares the alternatives.

Table 7.1 Alternatives and differentiation

Alternative	Strength	Limitation	Differentiation of the integrated solution
Manual scouting and canal rotation	No equipment; familiar	Subjective, late, wasteful	Evidence-based, early, field-level
Threshold irrigation systems	Can save roughly 20–30% of water [3]	Reactive; ignores growth stage	Stage-aware scheduling shared with disease and pest context
Mobile leaf-diagnosis applications	Easy to use	Mostly global data; after symptoms [4]	Adds 7–14 day risk forecasting and local thresholds
Standalone IoT pest traps	Automate counts	Alarm at high counts; no forecast [5]	Hatching-window forecasting with weather and treatment scheduling
Crop simulators and weather-only ML	Mechanistic or accurate at regional scale [17]	Need calibration; regional, not field-aware	Plot-level live soil data with Sri Lankan varieties
Separate point solutions	Specialised	No shared context; repeated sensing	One backbone, shared context, one bilingual interface

7.3 Business model and feasibility

The component theses converge on a tiered model. A **free tier** offers a freemium mobile application with basic image-based disease detection and general weather alerts in local languages. A **subscription tier** offers premium seasonal forecasts, stage-aware irrigation schedules and pest-mitigation plans; an indicative farmer subscription of about LKR 500 per month is proposed to support notifications, synchronisation and model updates [3]. An **institutional tier** offers district-level dashboards and analytics for cooperatives, extension services and RRDI researchers. Optional sensor bundles are offered to farms that want field-level monitoring. Hardware is kept inexpensive by using off-the-shelf ESP32, DFRobot-compatible and Raspberry Pi components, which allows staged funding and cluster purchasing by farmer cooperatives.

Likely channels are farmer organisations, agrarian service centres and agritech integrators, with RRDI and the Department of Agriculture as technical partners. Key resources are the field hardware, the cloud services, the trained models and curated data, and a small maintenance and support team.

Table 7.2 combines the pilot budgets reported in the component theses. Together they show that the pilot was affordable at roughly half a million rupees.

Table 7.2 Combined pilot budget from the component theses

Component	Main items	Pilot budget (LKR)	Source
Yield forecasting	3 sensor stations, weather station, ESP32-CAM backup, effort, field testing	132,000 – 140,000	[6]
Irrigation optimisation	3 sensor node bundles, cloud/Firebase	80,000	[3]
Disease detection and risk forecasting	3 Raspberry Pi 4, 3 USB cameras, cloud GPU (3 months), localisation	235,000	[4]
Pest management	4 ESP32-CAM modules, lures and housings, cloud, effort and field testing	80,000 – 88,000	[5]
Combined (sum of component budgets)		527,000 – 543,000	Sum of the above

These figures are pilot budgets, not a commercial cost model. Two points should be checked: the yield and irrigation budgets each include soil sensors, so; and commercial deployment would also require cloud hosting, support and replacement of hardware, for which the component theses give illustrative planning figures (for example LKR

1.2 million per year for cloud services and LKR 4.8 million for development in the yield thesis [6]). Technical feasibility is supported by the use of mature open-source tools and low-cost devices; operational feasibility depends on connectivity, farmer adoption and maintenance, as discussed in Section 7.4.

7.4 Adoption intellectual property and compliance

Adoption barriers include limited smartphone access and connectivity, trust in automated advice, language and literacy, the up-front cost of sensors, the need for seasonal retraining, and the effort of entering field events. The design responds with Sinhala/English delivery, a manual override, offline buffering, staged and cluster funding, and officer support. Pilot results on adoption are in Section 6.4.

Intellectual property. Potential intellectual property lies in the trained models, the curated local data sets, the feature-engineering and integration logic, and the localised interface design. At this stage these are treated as future protectable assets, not as confirmed filings. RRDI data are used under the terms agreed with the institute and are not redistributed.

Data protection and ethics. Participation is voluntary and based on informed consent; records are anonymised; imagery and farm data are access-controlled; advice is shown with confidence levels and is advisory; and model limits (region, season, variety) are documented. The Personal Data Protection Act, No. 9 of 2022 [15] and institutional ethics requirements apply. **Safety:** chemical and biopesticide advice is drawn from the Department of Agriculture knowledge base and reviewed by agronomists for medium-risk cases, and valve actuation is supervised.

7.5 Sustainable Development Goal contribution

The most plausible contribution is to SDG 2 (Zero Hunger), targets concerned with sustainable food production and resilient agricultural practices, through earlier and better-informed decisions that reduce avoidable loss. Contributions to SDG 6 (water-use efficiency) and SDG 12 (more targeted use of agrochemicals) follow from the irrigation, disease and pest components, and SDG 13 from adaptation to climate variability. These contributions are plausible consequences of the design; they are not

measured outcomes of this project, apart from what the pilot evidence in Chapter 6 shows .

Possible negative effects should be managed. Sensors, batteries and cameras create electronic waste, so repair, reuse and recycling arrangements are needed. Digital tools can widen inequality if poorer or older farmers cannot use them, which argues for cooperative ownership, officer-assisted access and low-cost tiers. Incorrect advice could cause loss, so confidence levels, expert review and manual override are essential. Over-reliance on automation could erode farmer knowledge, so the platform is designed to support rather than replace the farmer's judgement. Benefits are sustained by keeping models retrained on local data, funding maintenance through subscriptions and institutional partners, and involving cooperatives and extension services.

8 CONCLUSION AND FUTURE DIRECTIONS

8.1 Overall conclusion

The common problem was that smallholder paddy farmers lack affordable, local, timely and joined-up decision support for yield, irrigation, disease and pests. The team's answer is an integrated platform in which four components, a yield forecaster, a stage-aware irrigation scheduler, a disease detector and outbreak forecaster, and a pest monitor with hatching forecasts, share one data backbone, exchange the context that links them and deliver advice in Sinhala and English. The design shows that the four subproblems can be addressed by non-overlapping components whose interdependencies are handled through explicit, versioned interfaces rather than by duplicating sensing or merging code.

8.2 Principal integrated contribution

Beyond the four components, the team produced: a reference architecture for smallholder paddy decision support with a shared gateway and common data contracts; a catalogue of ten cross-component exchanges showing how soil wetness, spray windows, disease and pest risk and irrigation history improve one another's recommendations; an evaluation framework that combines component metrics, interface and fault-injection tests and farmer validation; and a reusable low-cost hardware and software pattern for field capture with offline tolerance. The main lesson is that the value of such a platform lies as much in how the components exchange context as in the accuracy of each one.

8.3 Future directions

In order of priority:

- **Stronger evidence.** Run multi-season, multi-district pilots with more nodes, and compare the platform with and without cross-component exchanges to measure their benefit.
- **Unify delivery.** Merge the farmer-facing front ends into one application and consolidate the data stores behind the gateway.

- **Deployment and adoption.** Partner with farmer cooperatives, agrarian service centres and RRDI; test the subscription model; plan hardware maintenance and recycling.
- **Wider coverage.** Add varieties, diseases and pests, the Maha season and other districts; consider other crops.
- **Better models and uncertainty.** Add calibrated probabilistic forecasts, remote-sensing and evapotranspiration features, and continuous learning from farmer feedback.
- **Supervised automation.** Move from advisory recommendations towards supervised valve actuation once reliability is shown.

REFERENCES

[1] V. Amaratunga, L. Wickramasinghe, A. Perera, J. Jayasinghe, and U. Rathnayake, "Artificial neural network to estimate the paddy yield prediction using climatic data," *Math. Probl. Eng.*, vol. 2020, Art. no. 8627824, 2020, doi: 10.1155/2020/8627824.

[2] P. Ekanayake, W. Rankothge, R. Weliwatta, and J. M. J. W. Jayasinghe, "Machine learning modelling of the relationship between weather and paddy yield in Sri Lanka," *J. Math.*, vol. 2021, Art. no. 9941899, 2021, doi: 10.1155/2021/9941899.

[3] J. M. S. U. I. Bandara, "Paddy irrigation optimization component of an IoT and AI-based smart agriculture platform," B.Sc. (Hons) thesis, Dept. Inf. Technol., Sri Lanka Inst. Inf. Technol., Sri Lanka, 2026.

[4] A. P. N. Satyana, "Disease detection and risk forecasting component of an IoT and AI-based smart agriculture platform," B.Sc. (Hons) thesis, Dept. Inf. Technol., Sri Lanka Inst. Inf. Technol., Sri Lanka, 2026.

[5] P. D. M. Kumarasinghe, "Pest management component of an IoT and AI-based smart agriculture platform," B.Sc. (Hons) thesis, Dept. Inf. Technol., Sri Lanka Inst. Inf. Technol., Sri Lanka, 2026.

[6] V. G. S. M. Siriwardhana, "Paddy yield forecasting component of an IoT and AI-based smart agriculture platform," B.Sc. (Hons) thesis, Dept. Inf. Technol., Sri Lanka Inst. Inf. Technol., Sri Lanka, 2026.

[7] NASA Langley Research Center, "POWER data access viewer and API." [Online]. Available: https://power.larc.nasa.gov/

[8] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 785–794.

[9] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[10] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[11] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.

[12] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[13] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[14] ISO/IEC 25010:2011, *Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models*, 2011.

[15] Parliament of Sri Lanka, "Personal Data Protection Act, No. 9 of 2022," Colombo, Sri Lanka, 2022.

[16] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Q.*, vol. 28, no. 1, pp. 75–105, 2004.

[17] D. P. Holzworth et al., "APSIM – Evolution towards a new generation of agricultural systems simulation," *Environ. Model. Softw.*, vol. 62, pp. 327–350, 2014.

APPENDICES

Supplementary material cited in this report is provided in Appendices A and B.

Appendix A Integrated interface specification

Table A.1 specifies the exchanges of Table 3.5. Payloads use the common envelope of Section 4.2; the body fields below are indicative and should be confirmed against the implemented APIs.

Table A.1 Integrated interface specification

No.	Producer → consumer	Body fields (indicative)	Trigger and frequency	Validation and error handling
1	Irrigation → Yield	irrigation_events, water_stress_index, growth_stage	Daily batch and on new event	Stage in allowed set; missing values flagged; consumer uses last valid
2	Yield → Irrigation	yield_risk, forecast_t_ha, confidence	Daily batch	Risk in 0–1; stale flag if older than one day
3	Irrigation → Disease	soil_moisture, irrigation_events	Hourly or on sensor upload	Range check 0–100%; buffered if offline
4	Disease → Irrigation	alert_level, spray_window_start, spray_window_end	On alert change	Window end after start; irrigation keeps manual override
5	Disease → Yield	disease_class, severity, risk_7d, risk_14d	On new image or daily	Probabilities in 0–1; missing treated as no signal
6	Pest → Yield	pest_risk_level, trap_temperature, trap_humidity	On new trap image	Level in {Low, Medium, High}; confidence adjusted only if valid
7	Pest → Irrigation	treatment_day, spray_schedule	On schedule change	Date in future; irrigation avoids wash-off windows
8	Irrigation → Pest	field_humidity, soil_condition	Daily	Range checks; last valid used if missing
9	Disease → Pest	disease_present, alert_level	On alert change	Boolean and level validated
10	Yield → Pest	crop_stress_level	Daily	Level in allowed set

Shared units: soil moisture in percent; EC in dS/m; temperature in °C; rainfall in mm; yield in t/ha; time as ISO 8601 with time zone; identifiers anonymised.

Appendix B Team contribution declaration

The following record of individual and shared contributions is to be agreed and confirmed by all four members .

Member	Individual contribution
V.G.S.M. Siriwardhana (IT22189158)	Paddy yield forecasting component
J.M.S.U.I. Bandara (IT22269348)	Paddy irrigation optimization component
A.P.N. Satyana (IT22205506)	Disease detection and risk forecasting component
P.D.M. Kumarasinghe (IT22254566)	Pest management component

Use of AI tools. Generative AI (Claude, Anthropic) was used to assemble this common report from the four individual theses and the common report template: summarising and organising the component content, drafting prose, creating the architecture diagrams and checking selected references. No experimental results, data or code were generated by the tool. Placeholders in the report mark information that the team must supply.